

cw1 submission

Regular languages

Recall that in the lectures we showed that the class of regular languages was closed under union, sequential composition, and Kleene closure.

The complement of a language L , written $\bar{L}$, is every string not in L , i.e. $\Sigma^* \setminus L$. Show that the regular languages are closed under complement.

Any regular language can be represented by a deterministic finite automata. Then, we can simply use the rejecting states of our FDA as the accepting states and vice-versa.

$$M' = (Q, \Sigma, \delta, q_0, Q \setminus F)$$

Also, it's important that our δ transition function is **complete**, otherwise we could reject sentences because of a missing transition.

Using (a), or other arguments, show that the regular languages are closed under intersection. The intersection of two languages $L_1 \cap L_2$ is the set of all strings that are in both L_1 and L_2 , that is $\{w \mid w \in L_1 \wedge w \in L_2\}$

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$

and we know that the regular languages are closed under the union and the complement.

An all-NFA is a variant of an NFA where a string w is only accepted if all states reached on word w are final, i.e. $\delta^*(q_0, w) \subseteq F$. Show that there is an all-NFA that recognises a language if and only if it is regular.

Let's assume that there is a language L_1 represented by this all-NFA. The complement of L_1 can be represented by this NFA and accepting if at least one path is a rejecting path. Now we construct NFA $\bar{L}_1$ by flipping the rejecting and the accepting states as we did in question a), then we obtain a language $\bar{L}_1^C$ that can be represented by a NFA, therefore a regular language. Because the complement of a regular language is also regular, then L_1 is regular.

Let's assume that a language L_2 is regular. Then it can be represented by a DFA, which is itself an all-NFA.

Therefore, an all-NFA recognizes a language if and only if it's regular.

Prove that $L = \{0^n 1^m 2^{m-n} \mid m \geq n \geq 0\}$ is not regular. You may use any of the three methods used in lectures: the Pumping Lemma, the Myhill-Nerode

theorem, or using (or proving) closure properties of the regular languages to reduce the problem to a known non-regular language. As an extension exercise, you may wish to try multiple methods.

We can use the pumping lemma:

- let's start from the string: $0^{p-1}1^p2$
- because $|xy| \leq p$, $|y| > 0$, we can have:
 - x is only zeros, and y is both 0s and 1s $x = 0^k$, $y = 0^{p-k-1}1^j$, $z = 1^{p-j}2$
 - x is only zeros and y is only 1s, $x = 0^{p-1}$, $y = 1$, $z = 1^{p-1}2$
 - y is only zeros, and therefore pumping y would increase n

in both cases, we cannot pump y , because we will only have one 2 at the end that depends on the number of zeros and ones, and 2 is necessarily part of z .

We can also use Myhill-Nerode:

- if we take $0^i1^j2^{m-n-1}$, $i \geq j > 1$, and we append $z = 2$ at the end, we always get a valid word.
- but using $0^i1^j2^{m-n-2}$, $i \geq j > 1$, and by appending $z = 2$, we always get an invalid one.

Context-free languages

Consider the CFG $G: S \rightarrow aS \mid aSbS \mid \varepsilon$

Informally characterise $L(G)$.

- a
- aaba
- aba
- aaaaaaaaaababa

A sequence of a and b , such that each b is preceded by one or more a 's.

Show that it is ambiguous by finding a string for which you can construct two parse trees and two leftmost derivations.

aaba

$S \rightarrow aSbS \rightarrow a(aS)bS \rightarrow aabaS \rightarrow aaba$

$S \rightarrow aS \rightarrow a(aSbS) \rightarrow aab(aS) \rightarrow aaba$

Find an unambiguous grammar for $L(G)$.

$$T \rightarrow aSb \mid a$$

$$S \rightarrow TS \mid \varepsilon$$

Give a pushdown automaton that recognises $L(G)$.

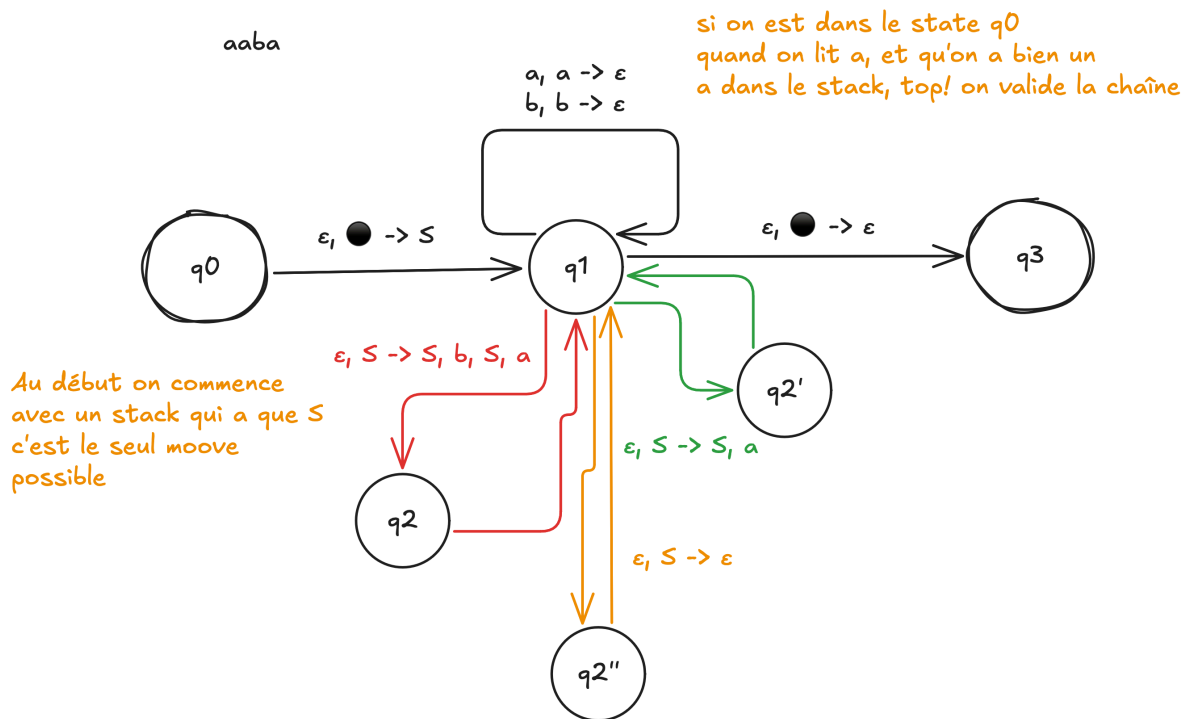


image-19.png

Some $w \in \{a, b\}^*$ have unique parse trees in G . What are those strings w ? Give an efficient test to tell whether w has this property. The test “try all parse trees to see how many yield w ” is not adequately efficient.

When a b can be attached to several a 's.

```
balance = 0
ambiguous = False
for i, sym in enumerate(w):
    if sym == 'a':
        balance += 1
    else if sym == 'b':
        if balance == 0:
            reject() # invalid
        balance -= 1
    # A b can attach to any of the open a's, it's ambiguous
    if balance >= 1:
        ambiguous = True
```

Reductions for undecidability

In lectures, we considered the Halting Problem H , the Looping Problem L , and the Uniform Halting Problem UH . Now we consider the Universal Looping problem UL : given a machine M , does M loop on all inputs R ?

Show, by reduction from L , that UL is undecidable.

We build a machine M' that ignores its input, writes w on the tape, and calls M . If M loops on w , then M' loops on all inputs as well. And we have no way of knowing whether M loops on w , which is the original problem.

Show, by constructing a suitable machine, that UL is co-semi-decidable. (Hint: interleaving.)

We can use the same thing as we saw for enumerating the machines for the halting problem:

```
Machine S takes as input  $\langle M \rangle$ :
  Enumerate all possible inputs  $x_1, x_2, x_3, \dots$ 
  For stage  $i = 1, 2, 3, \dots$ 
    For each  $j \leq i$ :
      Simulate one more step of  $M(x_j)$ 
      If  $M(x_j)$  halts at this step:
        OUTPUT "M does not loop on all inputs" HALT
```

Construct a reduction from H to Fac , and so show that Fac is undecidable.

Let's build a FAC machine that:

- takes parameter n
- simulates M on w , and:
- if it stops, calculates $n!$
- if it does not stop, and it does not stop either